Olwen Pichard
Lisa Le Goff
Baptiste De Paul
Yann Butter

Compte-rendu TP11

Spécification et Modélisation de Programme

L'objectif de ce TP est de reproduire le jeu de société « Les Aventuriers du rail » en C++, selon un cahier des charges bien défini : 2 à 4 joueurs, gestion complète des tours, des wagons, des cartes et des tickets destination.
Ce projet nous permet aussi d'utiliser et de mieux comprendre les outils du développement logiciel en équipe tels que le versionning avec Git, la compilation avec CMake, la documentation avec Doxyge.
En plus des éléments du cahier des charges, nous avons décidé d'ajouter une interface graphique en Qt6 (à l'aide de l'IA), afin d'avoir une meilleure visualisation de l'avancée de notre projet, et de le rendre plus concret.
Nous sommes une équipe de 4 personnes pour réaliser ce projet.

1. Création du diagramme de classe.

La création du diagramme de classe est la première tâche à accomplir concernant ce projet. En effet, il permet de définir chaque classe, de comprendre les relations entre chacune d'elles, ainsi que leur composition.
Pour notre projet, nous avons regroupé ces classes en 5 modules, documentés dans Doxygen via @ingroup.

Groupe	Classe
Modèle du jeu	Plateau, Ville, Route
Joueurs	Joueur
Cartes et Ticket	CarteTrain, Ticket
Couleurs (enums)	CouleurCarte, CouleurRoute, CouleurWagon
Interface Qt	Affichage (hérite de QMainWindow)

Les relations clés entre les classes sont :

- Plateau ♦→ Route : le plateau contient toutes les routes (agrégation).
- Plateau ♦→ Ville, CarteTrain : le plateau stocke les villes, et la pioche de CarteTrain.
- Plateau ←♦ Affichage : l'affichage content le plateau de jeu.
- Joueur ←♦ Route : la route contient un joueur.
- Joueur <-- Route : la route utilise un joueur
- Joueur --> CouleurWagon : un joueur utilise une couleur de Wagon (enum)
- Joueur ♦→ CarteTrain : un joueur contient des cartes Train
- Ville →♦ Ticket : les tickets contiennent des villes

- Affichage --> Joueur : l'affichage utilise des joueurs
- CarteTrain --> CouleurCarte : les cartes Train ont des couleurs.
- Route --> CouleurRoute : une route a une couleur spécifique.

Joueur et Route se référencent mutuellement. Pour éviter les inclusions circulaires, nous avons utilisé des forward declarations. Nous déclarons donc la classe sans l'inclure, ce qui suffit pour utiliser un pointeur.

Voici le diagramme de classe créé afin d'implémenter le jeu :

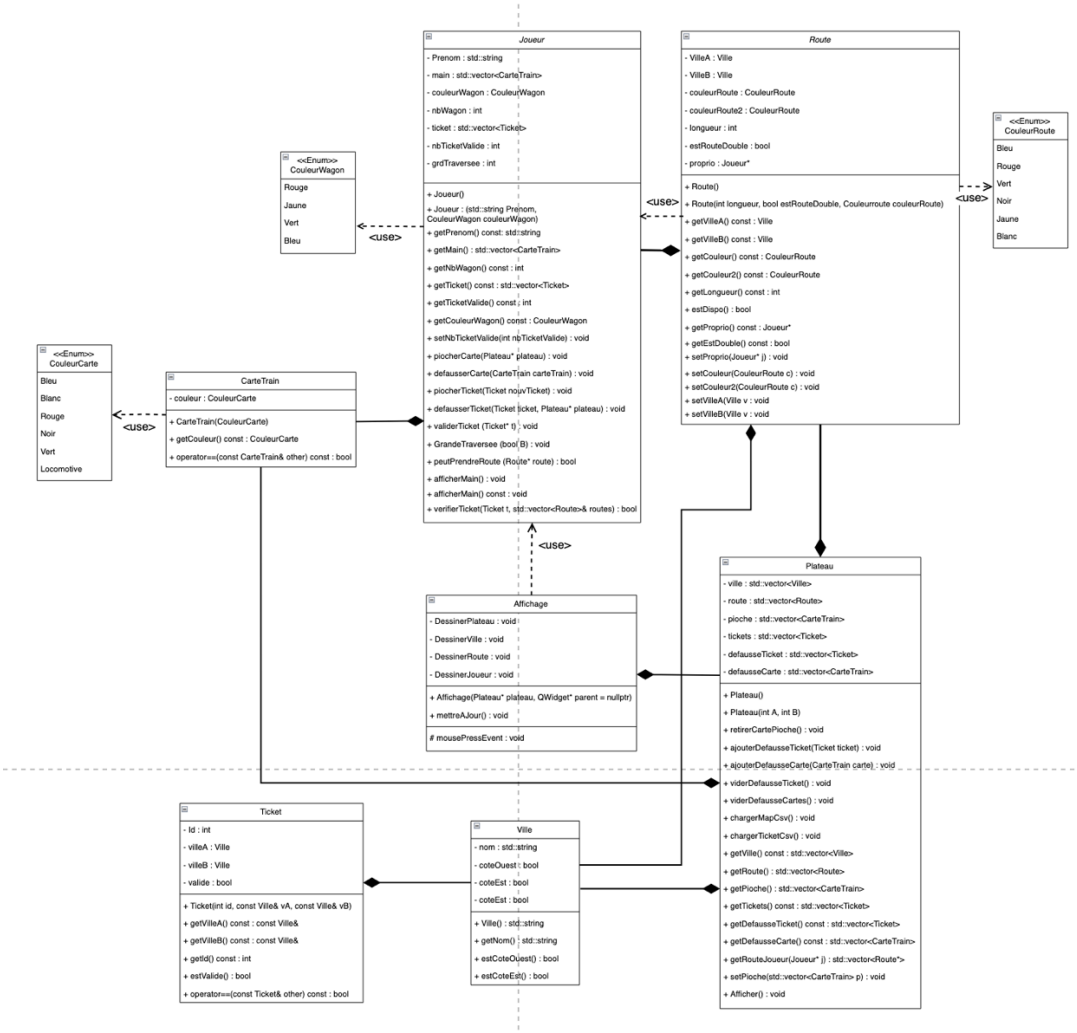

Figure 1 : Diagramme UML proposé.

2. Implémentation

Les routes et les tickets sont chargés deuis des fichiers CSV dès le démarrage de la partie, conformément au cahier des charges. Nous avons également créé un fichier CSV contenant les villes, afin de pouvoir les implémenter dans l'affichage et avoir un rendu visuel des routes prises et disponibles.

L'initialisation d'une partie se déroule comme suit :

- Saisie du nombre de joueurs (2 à 4) et de leur pseudo
- Attribution automatique d'une couleur de wagon par joueur (Rouge, Bleu, Jaune, Verte)
- Construction de la pioche : 6 couleurs * 10 cartes + 12 locomotives = 72 cartes
- Mélange de la pioche avec l'algorithme de Fisher-Yates via rand() + srand(time)
- Distribution de 4 cartes train à chaque joueur
- Mélange et distribution de 2 tickets destination à chaque joueur

La boucle de jeu tourne jusqu'à ce qu'un joueur valide son 6ème ticket ou n'ait plus de wagons. A chaque tour, le joueur courant est determiné par *nombredeTour % nbJoueurs*.

Trois actions sont disponibles à chaque tour :

- Action 1 : Piocher deux cartes train depuis la pioche.
- Action 2 : Poser des wagons sur une route disponible.
- Action 3 : Échanger ses 2 tickets contre deux nouveaux.

La prise d'une voie ferrée avec l'actions 2 est la plus complexe. En effet, voici ce qu'elle fait :

- Affichage de toutes les routes disponibles (estDispo() == true) avec leur index.
- Comptage des cartes par couleur dans la main + nombre de locomotives.
- Vérification des couleurs jouables pour la route choisie (voie 1 et voie 2 si double).
- Si plusieurs couleurs possibles, le joueur choisit laquelle utiliser

3. Création de la classe Animal

La classe Animal possède deux indicateurs d'accès : *protected* et *public*.

Dans l'indicateurs d'accès protected, nous retrouvons plusieurs variables telles que :

- Une chaîne de caractères pour nom de l'animal
- Un entier pour la position en abscisse de l'animal
- Un entier pour la position en ordonnées de l'animal
- Un booléen indiquant l'état de vie de l'animal
- Une variable de type Attaque qui indique le type d'attaque de l'animal
- Un entier qui définit la limite maximale du plateau en abscisse
- Un entier qui définit la limite maximale du plateau en ordonnées

Dans l'indicateur d'accès public, nous avons :

- Un constructeur Animal qui prend en paramètre deux entiers qui définiront les limites du plateau. Ce constructeur permet d'initialiser les limites du plateau de jeu, d'initialiser l'animal en vie, et de le positionner aléatoirement sur le plateau.
- Un destructeur virtuel qui permettra de détruire des objets de la classe Animal, comme lorsqu'un animal perd contre un autre.

- Une méthode qui prend en paramètre une référence vers un autre animal et retourne un booléen. Cette méthode récupère le type d'attaque courant en appelant l'accesseur getAttaque() et stocker le résultat dans une variable de type Attaque. Aussi, elle récupère le type d'attaque de l'animal passé en paramètre en appelant l'accesseur getAttaque() et stocker le résultat dans une variable de type Attaque. Enfin, elle retournera un booléen qui indiquera si l'attaque de l'animal courant gagne contre l'attaque de l'animal passé en paramètre.
- Une méthode virtuelle de « deplace(int maxX, int maxY) »
- Un accesseur qui renvoie une chaîne de caractère correspondant au nom de l'animal
- Un accesseur qui renvoie un entier correspondant à la position en x de l'animal
- Un accesseur qui renvoie un entier correspondant à la position en y de l'animal
- Un accesseur qui renvoie un booléen indiquant si l'animal est en vie ou non
- Un accesseur qui renvoie un objet de type Attaque correspondant au type d'attaque de l'animal
- Un mutateur qui prend en paramètre un booléen et indique si l'animal est vivant ou non, et modifie le paramètre vivant de l'animal en conséquence
- Un virtuel pur de setAttaque().

4. Vérification automatique des tickets

Après chaque pose de wagons, le programme vérifie automatiquement si un ticket est complété. On utilise un algorithme BFS (Breadth-First Search, parcours en largeur) :

- Construction d'un graphe avec uniquement les routes appartenant au joueur
- Parcours en largeur depuis la ville de départ du ticket
- Si la ville d'arrivée est atteinte → ticket validé, retiré de la main, nouveau ticket pioché

5. Recyclage de la pioche de tickets

Quand la pioche de tickets est vide, on récupère les tickets défaussés, on les mélange, et on les remet en jeu.

6. Création du main et jeu

Dans cette dernière partie, nous pouvons utiliser les différentes classes créent pour tester et jouer au jeu de plateau.
Pour cela, nous créons tout d'abord une fonction « afficherPlateau » qui permettra d'afficher les limites de chaque case, et de placer les animaux dans leur case respective.
La fonction resoudreConflit permettra de supprimer les animaux qui ne seront plus en vie après une attaque.

La fonction compterVivant permet d'afficher le nombre d'animaux toujours en vie sur le plateau.
Enfin, dans le main(), nous utilisons toutes ces fonctions pour construire le jeu et l'implémenter selon les règles pré-étables. Une manche recommencera à chaque fois que l'utilisateur écrira « o » ou « 0 ».

7. Outils de développement

8.1 Git et GitHub

Pour réaliser ce projet, nous avons travaillé sur des branches séparées aussi souvent que possible, même si n'était toujours le cas.
Dans la majorité des cas, nous réalisions ce processus :

- Branche individuelle pour travailler en parallèle sans se bloquer
- Résolution de conflits quand deux personnes modifient le même fichier
- *git checkout –ours* pour conserver sa version locale en cas d'urgence
- *Commit de chaque modification pour garder un historique lisible*

8.2 CMake

CMake génère les fichiers de compilation adaptés à chaque machine. Le même CMakeLists.txt fonctionne sur Linux, MacOS et Windows sans modification.

Nous avons utilisé :

- *find_package(Qt6)* et *find_package(Dowygen)* pour intégrer les dépendances automatiquement
- *option(BUILD_DOC_ONLY)* pour compiler la documentation sans Qt6 installé
- *configure_file()* substitue *@CMAKE_SOURCE_DIR@* et *@PROJECT_VERSION@* dans le Doxyfile
- Cible doc : *cmake –build . –target doc* pour générer la documentation
- Cible doc-open : génère et ouvre directement dans le navigateur

8.3 Doxygen

Toutes les classes, méthodes et attributs sont documentés avec des commentaires Doxygen. La documentation est générée en HTML avec des graphes de dépendances (Graphviz).

- @brief, @param, @return sur chaque méthode publique
- @ingroup pour regrouper les classes par module dans la doc
- modules.h centralise les @defgroup (Modèle, Joueurs, Cartes, Interface Qt)
- README.md utilisé comme page d'accueil (USE_MDFILE_AS_MAINPAGE)
- Graphes d'appels et diagrammes UML générés automatiquement avec Graphviz

8.4 Qt6

L'interface console était fonctionnelle et suffisante. Nous avons ajouté Qt6 pour visualiser le plateau en temps réel, ce qui n'était pas dans le cahier des charges mais permettait de mieux visualiser.

Nous avons utilisé :

- *QGraphicsScene* / *QGraphicsView* pour le dessin 2D
- Villes dessinées en cercles blancs avec nom en rouge centré
- Routes en segments rectangulaires orientés : angle calculé avec atan2(dy, dx), rotation appliquée avec *QTransform*
- Routes doubles : deuxième voie décalée perpendiculairement de 17 px
- Propriétaire affiché au centre de chaque route revendiquée
- *app.processEvents()* appelé dans la boucle console pour garder la fenêtre réactive

8. Ce que nous retenons

Ce qui nous a posé problème

- Les inclusions circulaires entre Joueur, Route et Plateau que nous avons résolu avec les forward declarations
- La synchronisation de l'interface Qt avec la logique console avec *app.processEvents()* dans la boucle
- Calculer les rotations des segments de routes avec atan2 pour l'angle, et QTransform pour l'appliquer
- Résoudre les conflits Git quand plusieurs personnes modifiaient les mêmes fichiers
- La répartition des charges de travail en équipe
- La gestion du temps

Ce que nous avons appris

- CMake : compilation propre et cross-plateforme, gestion des dépendances externes
- Doxygen : documenter tout le code. Il est préférable de le faire dès le début plutôt qu'à la fin.
- Git en équipe : les branches sont indispensables, les conflits sont gérables
- Qt6 / QGraphicsScene : rendu 2D, transformations géométriques
- La quantité de travail qu'implique un projet complet

9. Conclusion

Le jeu est fonctionnel, nous pouvons jouer à 2-4 joueurs, gérer les tours, poser des wagons, valider des tickets automatiquement, et voir le résultat en temps réel sur l'interface Qt. La plupart des fonctionnalités demandées dans le sujet est implémenté, hormis la grande traversée que nous n'avons pas réussi à faire.